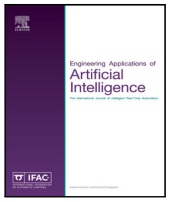
Engineering Applications of Artificial Intelligence 126 (2023) 106941



Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai





A deep reinforcement learning algorithm to control a two-wheeled scooter with a humanoid robot

Jacky Baltes, Guilherme Christmann, Saeed Saeedvand *

Department of Electrical Engineering, National Taiwan Normal University, Taipei, Taiwan

ARTICLE INFO

Keywords:
Deep reinforcement learning
Proximal policy optimization (PPO)
Two-wheeled vehicles
PID control
Humanoid robotics

ABSTRACT

Balancing a two-wheeled scooter is considered a challenging task for robots, as it is a non-linear control problem in a highly dynamic environment. The rapid pace of development of deep reinforcement learning has enabled robots to perform complex control tasks. In this paper, a deep reinforcement learning algorithm is proposed to learn the steering control of the scooter for balancing and patch tracking using an unmodified humanoid robot. Two control strategies are developed, analyzed, and compared: a classical Proportional–Integral–Derivative (PID) controller and a Deep Reinforcement Learning (DRL) controller based on Proximal Policy Optimization (PPO) algorithm. The ability of the robot to balance the scooter using both approaches is extensively evaluated. Challenging control scenarios are tested at low scooter speeds, including 2.5, 5, and 10 km/h. Steering velocities are also varied, including 10, 20, and 40 rad/s. The evaluations include upright balance without disturbances, upright balance under disturbances, tracking sinusoidal path, and path tracking. A 3D model of the humanoid robot and scooter system is developed, which is simulated in a state-of-the-art GPU-based simulation environment as a training and test bed (NVidia's Isaac Gym). Despite the fact that the PID controller successfully balances the robot, better final results are achieved with the proposed DRL. The results indicate a 52% improvement on average in different speeds with better performance in path tracking control. Controller command evaluation on the real robot and scooter indicates the robot's complete capability to realize steering control velocities.

1. Introduction

Incrementally increasing the set of skills that a robot can acquire and perform is a key aspect of developing general-purpose robots. Various sports and entertainment robots have been developed in the past, including remarkable humanoid robots that can ski, ice skate, rock climb, and perform magic tricks, Iverach-Brereton et al. (2012), Jeong et al. (2022) and Baltes et al. (2014). One of the fundamental attributes of humanoid robotics research is the versatility of the human shape, which makes focusing on using unmodified and general-purpose humanoid robots important. An ever-increasing range of skills and tasks will lead to more capable robots that can help humans in many different ways. In order to create robots capable of adapting to various environments and performing tasks originally designed for humans, they must possess different interactive systems as individual skills. In the robotics field, some areas of study, such as industrial robots, have seen great improvements year-by-year. However, industrial robots are developed with a single task in mind and are not easily repurposed for other tasks or environments. On the other hand, the lightning-quick pace of the development of Machine Learning (ML) systems like DRL techniques has enabled us to tackle (and solve) long-standing challenges in a wide variety of fields.

Considering the capabilities of the state-of-the-art in robotics and autonomous vehicles, it is noted that the operation of Two-Wheeled Vehicles (TWV) by robots has not received wide attention in the literature yet. TWVs, such as motorcycles and bicycles, provide an interesting problem and are valuable benchmarks regarding state-of-the-art in control. TWV, as opposed to Four-Wheeled Vehicles (FWV), are not stable at low speeds and are uncontrollable when not moving. This makes them a much harder problem than the control and path tracking of car-like and other FWV. To follow a desired path accurately while operating a TWV, the fusion of balance and orientation control is necessary. Furthermore, a high degree of precision is required to apply the correct steering and throttle commands.

The research area that tackles these kinds of problems as their primary goal is humanoid robotics (Saeedvand et al., 2019b). Like many robotics developments, humanoid robots have significantly improved hardware and software. Like many areas of AI, deep learning (DL) has greatly improved the current systems (Sünderhauf et al., 2018).

* Corresponding author.
E-mail address: saeedvand@ntnu.edu.tw (S. Saeedvand).

https://doi.org/10.1016/j.engappai.2023.106941
Received 10 November 2022; Received in revised form 15 June 2023; Accepted 3 August 2023
Available online 6 September 2023
0952-1976/© 2023 Elsevier Ltd. All rights reserved.

Most notably, reinforcement learning and deep reinforcement learning techniques have allowed robots to learn complex behaviors directly from raw perception data through a technique called end-to-end reinforcement learning (Lillicrap et al., 2015). In this paper, we continue our previous work on DRL algorithms (Saeedvand et al., 2021), and we present the development, evaluation, and comparison of two types of balancing controller algorithms for a humanoid robot driving a scooter: a PID controller as one of the most widely used types of control and our DRL-based controller. We use a large-sized THORMANG3 humanoid robot and a commercially available electric scooter manufactured by the Taiwanese Gogoro company (gogoro, 2022; Robotis, 2022), which we also proposed a web technologies-based simulator for simplifying installation of the software (Baltes et al., 2022), (see Fig. 1).

In this study, the effectiveness of classical Proportional–Integral–Derivative (PID) control is evaluated against a controller learned by a deep reinforcement learning (DRL) algorithm based on the model-free proximal policy optimization (PPO). The main contributions to this study are summarized as follows.

1. Proposing a scooter balance control by a general-purpose humanoid robot in the presence of external disturbances for the first time.
2. The proposed algorithm is used to track the path and balance the scooter at different speeds and steering velocities.
3. A 3D CAD model and simulation of a humanoid robot and electric scooter have been designed and released.
4. Extensive evaluation of the proposed algorithm (trained on the GPU), and comparison to the PID controller with and without external disturbances.

The main motivation for this research is the shortage of focus on controlling highly dynamic systems like scooters by multi-purpose robots. In this regard, the capabilities of humanoid robots to control scooters have been evaluated through two approaches, including low scooter speeds (2.5, 5, and 10 km/h) as hard scenarios to control. As an important element of scooter control, steering velocities are also varied (10, 20, and 40 rad/s) in the experiments. The evaluations include upright balance without disturbances, upright balance under disturbances, tracking Sinusoidal path, and path tracking. A 3D model of the humanoid robot and scooter system has been developed, which is simulated in a state-of-the-art GPU-based simulation environment called NVidia's Isaac Gym as training and testbed, NVIDIA (2022).

The rest of the paper is organized as follows. In Section 2, we discuss the literature review and the theory of fundamentals of balance control, including two-wheeled vehicle dynamics and reinforcement learning. Section 3 describes the Robot-Scooter System. Section 4 discusses the Proposed Methodology, including Balance Control, Direction Control — Path Tracking, PPO — Proximal Policy Optimization, Our DRL Agent based on PPO, and balance Control in detail. In Section 5, we discuss the experimental results. Finally, we conclude the paper in Section 6.

2. Literature review

In this section, we present the literature review and the theory of fundamentals of balance control based on the two-wheeled vehicle models, bicycles, and scooters. Also, we present the background of reinforcement learning methods and related literature.

2.1. Two-wheeled vehicle dynamics

In contrast to four (or more) wheeled vehicles, two-wheeled vehicles are not self-stable, especially at low speeds (assuming standard configurations). When moving slowly, TWV requires continuous and quick input to maintain stability (Singhania et al., 2019). Moreover, due to the dynamics of the system, whenever a steering command is applied (e.g. by turning the handle-bar), the whole system is subjected to forces that affect the overall balance. Because of this, the control of even simple actions, such as making a turn, becomes a complex problem, which is juggling between tracking errors and balancing stability.

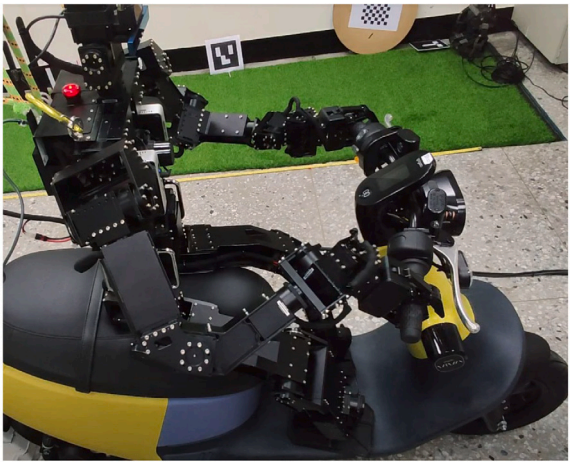


Fig. 1. THORMANG3 robot mounted on the Gogoro electric scooter.

In the literature, there are three main categories of balancing controllers (Vu and Nguyen, 2020). Steering-based controllers, which aim to achieve stability by controlling the angle of the steering bar handle (Tanaka and Murakami, 2004; Wang et al., 2012); (ii) Manipulating the center of mass, which usually consists in installing a balancing subsystem that can manipulate a rod of mass or similar devices (Lee and Ham, 2002; Yamakita et al., 2006), and (iii) systems that make use of a flywheel (Kim et al., 2015; Sikander and Prasad, 2019) as one important and relevant to the research community. The robot-scooter system could be considered a control of the center of mass, where the robot's upper body is analogous to the rod of mass in such systems. However, our humanoid robot in the experiments is able to perform the quick sideways movement as is common in this type of controller. Therefore, in this work, our focus specifically is on steering-based (i) controllers applied to a robot-scooter system that robot inverse kinematics can run it. In all of the controllers that are designed the only actuation is done directly in the steering joint.

In one of the simplest steering-based controller models, it can be assumed that the rider (a humanoid robot in our case) does not move. In this manner, it is possible to model the balancing problem as an inverted pendulum model, with the dynamics represented as the equilibrium of gravity and centrifugal forces. In this type of model, it is possible to design a steering controller to stabilize the vehicle such as in Tanaka and Murakami (2004). The derivation that follows below comes from Astrom et al. (2005), which is a great survey on the dynamics of bicycle systems and includes higher-order and non-linear models that are out of the scope of this work. Looking at Fig. 2, we can infer that the angular velocity around the vertical z axis is $\omega = V\delta/b$. Noting that the coordinate frame is relative to the bicycle model, the angular momentum balance can be computed as such:

$$J\frac{d^2\phi}{dt^2} - mgh\phi = \frac{DV}{b}\frac{d\delta}{dt} + \frac{mV^2h}{b}\delta \tag{1}$$

where J is the moment of inertia with respect to the x axis and $D = -J_{xz}$ the inertia product with respect to the xz axes, V is the linear velocity of a bicycle along the x axis, m is the total mass of the vehicle, b is the distance from the center of the front wheel to the center of the rear wheel, and h is the distance from the ground to the center of mass.

From this model, we can apply the simple control law $\delta = -k_1\phi$. Intuitively, this implies turning the steering handle to point in the same direction the bicycle is "falling". This makes the vehicle turn under its center of mass and stabilizes the system. The control law results in the following closed-loop system:

$$J\frac{d^2\phi}{dt^2} + \frac{DVk_1}{b}\frac{d\delta}{dt} + \left(\frac{mV^2hk_1}{b} - mgh\right)\phi = 0 \tag{2}$$

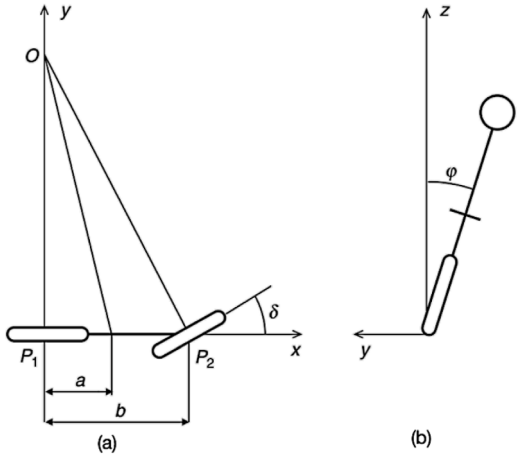


Fig. 2. Top (a) and rear (b) views of a simple bicycle model. Steering angle is δ, and the roll angle is ϕ (Astrom et al., 2005).

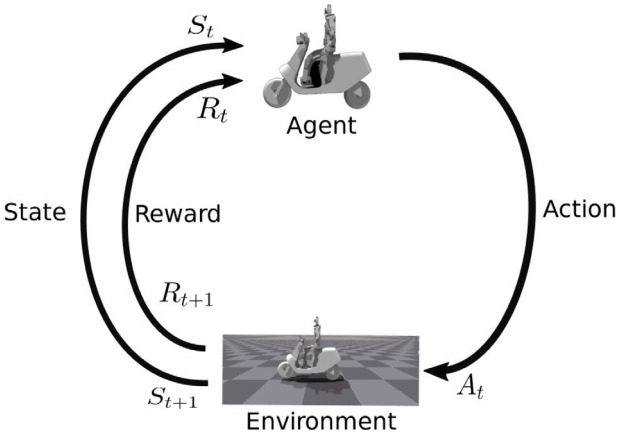


Fig. 3. A simplified diagram of the DRL agent loop.

which is stable if our gain $k_1 > bg/V^2$ (Astrom et al., 2005), suggesting that at higher speeds, stabilization is easier, and we can apply a smaller gain.

As previously mentioned, another common type of balancing control is by manipulating the center of mass. These usually consist of installing a controllable rod of mass, increasing the bicycle weight, and having a slow response time (Vu and Nguyen, 2020). The advantage is that, in contrast to steering-based controllers, these can perform balance control by moving at low speeds or even when completely stopped. An example of such a system, including its complete derivation, has been done by Utano and Yamakita (2005). In their work, the system consists of two links, one for the vehicle's frame and the wheels and a second representing the balance and its mass. The following dynamics equation is presented for the whole system:

$$\begin{bmatrix} M & J^T \\ J & 0 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ \lambda \end{bmatrix} = \begin{bmatrix} B \\ 0 \end{bmatrix} \tau + \begin{bmatrix} -H \\ -\dot{J}\dot{q} \end{bmatrix} \tag{3}$$

where q is a generalized coordinate vector, τ is the dual force vector for the coordinate vector, M is an inertia matrix, H represents Coriolis-centrifugal force, J is a constrained (that the wheel is fixed to the body, among others) Jacobian matrix, λ are the corresponding Lagrange multipliers and B represents other potential terms (Yamakita et al., 2006). This type of control system is especially relevant to our system since our humanoid robot can take the role of the "balancer" and perform motions such as leaning to the sides to affect the center of mass of the vehicle.

Given the types of controllers mentioned above, it is believed that an interesting engineering challenge would be posed by enabling humanoid robots to ride a scooter in a similar manner to humans. Similar works present in the literature are (Huang et al., 2017, 2018) in which a small-sized humanoid robot and a bicycle are specially constructed. Notably, in these works, the robot is capable of pedaling in addition to steering and balancing on the bike. In contrast, we propose enabling a large humanoid robot to ride a real electric scooter with as few modifications to the vehicle as possible. In comparison to well-acknowledged manuscripts (Huang et al., 2017, 2018), in this study, the focus is on a two-wheeled scooter balancing with an adult-sized humanoid robot is new and presents unique challenges, especially considering the challenging control scenarios tested at different scooter speeds and steering velocities. Besides, the extensive and advanced simulation environment, robust solution, and detailed analysis are also elements that extend beyond the scope of the previous works. We believe that our main challenges for the real robot lie in motion control: the accuracy of the arm and body motions to correctly perform steering commands while keeping a stable balance and following a desired path. In this work, all of the experiments are done in a simulation environment and directly controlling the steering joint angle.

2.2. Reinforcement learning

Reinforcement learning (RL) is a family of algorithms belonging to the field of machine learning, and it is a separate field of solving optimization problems, although both deal with optimization problems (Abd Elaziz et al., 2021; Abualigah et al., 2021b; Agushaka et al., 2022; Abualigah et al., 2021a; Saeedvand et al., 2019a; Messerschmidt and Engelbrecht, 2004). DRL differs from standard neural networks, which are the foundation of modern machine learning and neural network control based models (Zamfirache et al., 2023), or efficient Convolution Neural Networks (CNNs) (Verma et al., 2022) in several ways. Besides different control techniques exhibited for control problems, for instance, some studies used a polynomial control design with optimization-based techniques (Madiouni et al., 2019), or control the cart position by iterative feedback tuning algorithm (Roman et al., 2022) or Nonlinear Optimal Control solutions (Rigatos et al., 2017).

Neural nets interact with data primarily in an offline manner. The desired mappings from input to outputs are provided in batches, and the learning signal comes from the measured error in its predictions. DRL takes a different approach in the form of an agent interacting with an environment by performing actions. The performed actions cause changes to the environment and give rise to new states. The agent learns by receiving a reward signal, a scalar value, for each action that it takes and the resulting new state. By iterative interaction with the environment, the agent learns which sequence of actions provides the best reward in the long run. A simplified diagram of this feedback loop is presented in Fig. 3. The following elements can be identified in DRL methods:

- A policy — generates the behavior, i.e. actions, of the agent at any given state;
- A reward signal — determines how good a performed action is. It is the learning signal, and the goal of the agent is to maximize it;
- A value function — measures the expected reward for a specified state, the accumulated reward for the future;
- Model of environment, a model of the environment is used to plan and predict future states;

DRL is an extensive family of algorithms, and many methods have been explored since the 1980s, such as temporal difference methods and policy gradient methods, among many others (Sutton and Barto, 2018). With the popularization of deep learning, DRL has become prominent in recent years. Robotics is an especially interesting field

with plenty of potential applications (Sünderhauf et al., 2018). Because robots usually have many degrees of freedom, modeling an optimal control system can become complex. Optimal control approaches offer advantages over general non-optimal and non-learning control methods for balancing a two-wheeled scooter. These benefits include improved stability, efficiency, robustness against uncertainties, adaptability through learning, and optimized performance criteria. This makes it quite appropriate to try to propose optimal control approaches for our problem. Model-free DRL methods shine in this area, allowing agents to learn complex actions and emergent behavior. In the absence of a model of the world, modern methods combine the use of neural networks that learn to predict reward estimates for future states and actions. The neural networks essentially construct their own model during training, making it possible to learn good policies directly from high-dimensional sensor data such as images. These types of methods are usually called DRL.

DRL methods can be further divided into methods for discrete and continuous action spaces. An example of a method that works in discrete action spaces is Deep Q-Learning (DQL) (Mnih et al., 2013). It initiated the modern interest in deep DRL methods and consisted of using a neural network to map the raw pixel states to the reward expectation estimates for each possible action. While it demonstrated DRL can work for highly complex environments and input data, it does not perform well for continuous spaces. For continuous action spaces, methods such as the policy gradient algorithms are more commonly used and have been applied in the context of robotics even prior to the deep learning interest boom (Peters and Schaal, 2006). In recent years, many deep DRL methods based on traditional policy gradients have been researched, some specifically applied in robots and manipulation such as Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al., 2015). Other examples of methods from the continuous domain are actor-critic methods (A2C and A3C) (Mnih et al., 2016), trust region policy optimization (TRPO) (Schulman et al., 2015), and PPO (Schulman et al., 2017).

3. Robot-scooter system

This section details the objects of study of this work, namely the robot and the scooter. It presents the relevant specifications, physical properties, and other relevant information, such as reference frames.

3.1. The two-wheeled vehicle - Gogoro scooter

We use a two-Wheeled Vehicle that commercially is available Gogoro VIVA Plus scooter, depicted in Fig. 4. Scooters are a good choice for this problem because the acceleration can be controlled by the throttle on the handlebar. The shape of a scooter makes it easy to place the robot in a sitting position and easily reach, grasp, and manipulate the handlebar. A motorcycle would be a bigger challenge for robot placement.

A 3D model using the CAD software Autodesk Fusion 360 has been designed. In order to get an accurate simulation environment and be able to transfer control systems and policies over to the real world, the physical properties of the real scooter must be considered. The relevant specifications of the electric scooter are presented in Table 1, Fig. 5 depicts the center of gravity of the scooter, and the inertia matrix for the whole body, in kg m^2, is the following:

$$\begin{bmatrix} I_{xx} & I_{xy} & I_{xz} \\ I_{yx} & I_{yy} & I_{yz} \\ I_{zx} & I_{zy} & I_{zz} \end{bmatrix} = \begin{bmatrix} 4.40515 & 0.117584 & -1.66168 \\ -0.117584 & 18.0137 & 0.098875 \\ 1.66168 & -0.098875 & 14.8791 \end{bmatrix}$$



Fig. 4. Gogoro Viva Plus scooter.

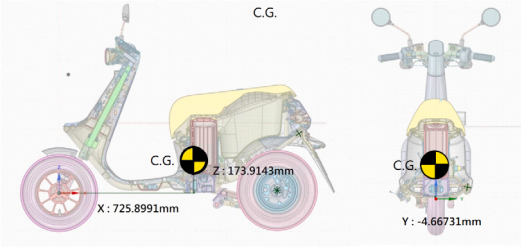


Fig. 5. Center of gravity of the scooter.

Table 1
Specifications of the Gogoro scooter.

Wheel base	1164 mm
Seat height	740 mm
Weight (with batteries)	79.2 kg
Max power output	3 kW @ 500 rpm
Max horsepower	4.02 hp @ 500 rpm
Max torque (Motor/Wheel)	115 Nm @ 200 rpm
Hill climb ability	20% (11°): 25 km/h 10% (6°): 40 km/h
Max lean angle	Left: 36° Right: 41°
Max riding range p/battery	Approx. 85 km

3.2. The adult-size humanoid robot

The adult-size humanoid robot used in this work is named Tactical Hazardous Operations Robot 3 (THORMANG3), presented in Fig. 6. It is a commercial large-sized biped humanoid robot manufactured by the Robotis company. THORMANG3 has two Intel NUC computers and 29 degrees of freedom. The complete specifications of the robot are presented in Table 2, and its system diagram in Fig. 8.

THORMANG3 is a biped robot capable of performing a wide range of motions, including grasping and manipulating objects. Most importantly, standing at just a bit over 137 cm of height, it is appropriately sized to sit on the scooter and grasp the steering handle in a similar manner to a human. A sequence depicting the grappling motion is shown in Fig. 7. The steering bar can be fully rotated in both directions, and the throttle can be controlled with a high degree of precision.

While the focus of this particular work is steering-based controllers in simulation, and directly manipulating the steering handle is not investigated, it is still important to keep the research grounded in reality as much as possible. The methods and results presented in this work set the groundwork and open up the paths for using the same robot in a more human-like manner in future research.



Fig. 6. THORMANG3 – Large-sized humanoid robot.

Table 2
Hardware and sensor specifications of THORMANG3.

Degrees of freedom	29
Actuator	200 W × 10/100 W × 11/20 W × 8
Computer	Intel NUC (i5 Processor) – 8 GB RAM DDR4 × 2
Wireless router	DLink DIR-806A
Camera	Logitech C920 HD Camera
LiDAR	Hokuyo UTM-30LX-EW
F/T sensor	ATi Mini58-SI-2800-120 × 2
IMU	MicroStrain 3DM-GX4-25
Battery	22 V, 22 000 mA–18.5 V, 11 000 mA
Height	137.5 cm
Weight	42 kg

3.3. Robot-scooter system

The resulting 3D models are presented in Fig. 9. The relevant axis of rotations is also presented. Fig. 9(a) shows the rear view of the model and the tilt angle, represented by the Greek letter ϕ. In later experiments, the robot is considered as having fallen over when $\phi > 65°$ since the system cannot recover from such an extreme angle. Fig. 9(b) shows a top-down view of the robot and the yaw (orientation) is represented by the Greek letter θ. In the experiments, $\dot{\theta}$ represents the rate of change, that is, the angular velocity of the robot for the yaw axis of rotation. The control variable of all of the experiments is the steering joint of the system, shown in orange. The steering joint is implemented as a revolute joint, with a maximum turning angle of 30° in either direction.

The sections that follow this detail the implementation of the PID controller and the DRL agent training. In both instances, the same model, written in URDF format, is utilized since both simulators fully support it. All described experiments are performed for the set of three velocities in the heading direction of the robot (θ). The velocities are 10 rad/s, 20 rad/s, and 40 rad/s. Considering that the radius of the wheel is 0.25 m, these correspond to 2.5 km/h, 5 km/h, and 10 km/h, respectively. There are a few noted exceptions that are discussed later. These are selected to investigate the stability of the system under different velocities. The velocities are relatively slow since under these circumstances TWV is the hardest to control and balance (Singhania et al., 2019).

4. Proposed methodology

This section focuses on the implementation details of the PID and proposed deep reinforcement learning steering-based controllers. It also details the experiments realized for each controller.

4.1. PID controller

PID controllers are a staple of the control field and are one of the most widely used types of control. The PID controller experiments, including balance and tracking control, have been evaluated using the Gazebo simulator Fig. 10, which is a popular simulation environment for robotics.

In the following sections we present the implementation details for the balance control and path tracking. In both controllers the end effector and output of the robot's inverse kinematics, the implementations consider the actuated steering joint.

4.1.1. Balance control

The balance control is a simple controller based on the simplistic bicycle model presented earlier in Eq. (1). The IMU data for the current tilt angle of the robot (ϕ) is read at a rate of 10 Hz. The error signal is calculated from the difference between a desired and the current tilt angle: Error $= \phi - \phi_{\text{ref}}$. By setting the ϕ_{ref} to values different from 0 it is possible to realize turning motions. In these situations, the controller works to keep a stable balance with a tilt angle. Because of the dynamics of the system, the scooter turns.

To analyze the robustness of the control and response to impulses, lateral forces of increasing magnitude are generated in the robot's center of mass. The forces are generated using the simulator's physics engine and alternated between each side of the system. The following experiments have been realized for balance control with the PID controllers:

1. Balance under no disturbances with $\phi_{\text{ref}} = 0$, i.e. keeping the robot upright;
2. Balance while turning ($\phi_{\text{ref}} \neq 0$). The reference starts at 0 and is set to a negative value and positive value after some time;
3. Balance under increasing disturbances with $\phi_{\text{ref}} = 0$.

4.1.2. Direction control – Path tracking

As previously explained, the system is able to perform turns when setting a $\phi_{\text{ref}} \neq 0$ and tracking it in a stable manner. In order to track a desired path, a second PID controller is designed on top of the balance PID controller. This second controller is called the reference controller and takes care of setting the ϕ_{ref} so that it can follow a determined path.

The path is made of points placed in the 2D ground plane. The tracking control is based on figuring out the correct orientation in order to align with the path. This is done by first finding the point closest to the robot-scooter system (the reference frame fixed on the rear wheel). Trying to align with a point that is too close would not work, as it would generate motions of large magnitude and instability. Instead, a look ahead distance is defined as 7 m. The orientation error at any time-step is computed as the difference between the current orientation θ of the robot and the angle between the robot and the point 7 m along the path, as shown in Fig. 11. The orientation error is computed by the equation $\tilde{theta} = \theta - \theta_{ref}$, where the second term is the angle between the current position and the look-ahead point. Different forward distances have been experimented and 7 is chosen because it performs well for all velocities. The reference controller works in tandem with the balance controller and has its own set of gains which are presented in Table 6.

The path controller behavior has been analyzed for two distinct paths. The first path is a straight line 10 m away from the robot-scooter system's initial position. By making the initial orientation perpendicular to the path it is possible to observe the convergence behavior as well as problems such as overshooting and oscillations. The second path is a sinusoidal-shaped path. One full cycle of the sinusoidal path spanned 80 m in the x axis and had a magnitude of 6 m. The PID path controller is subject to the following experiments:

1. Tracking two paths: a straight line and a sinusoidal, under no disturbances.

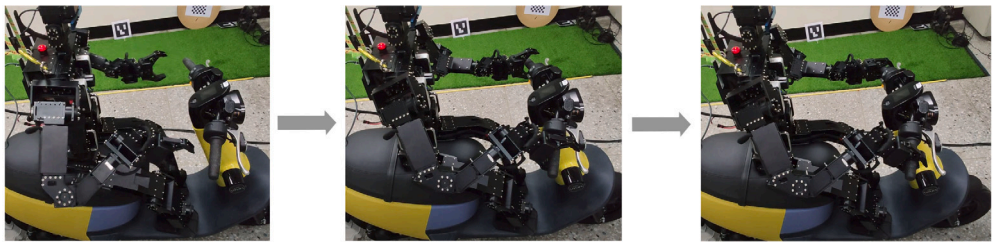


Fig. 7. Robot's steering handle grasping sequence from left to right.

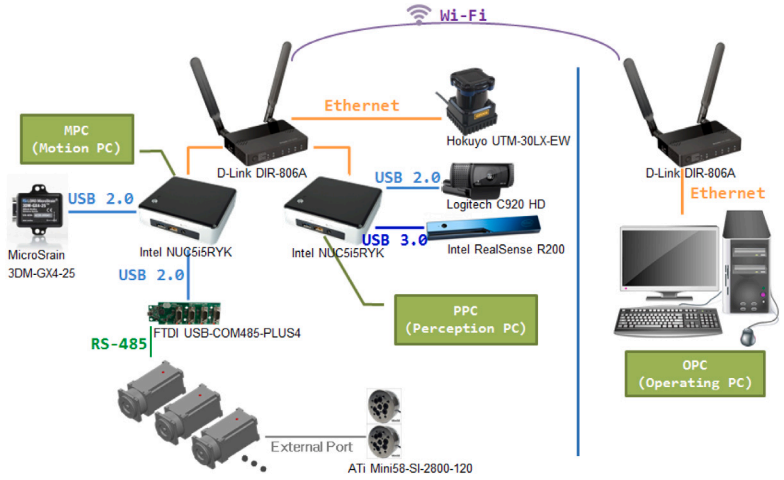
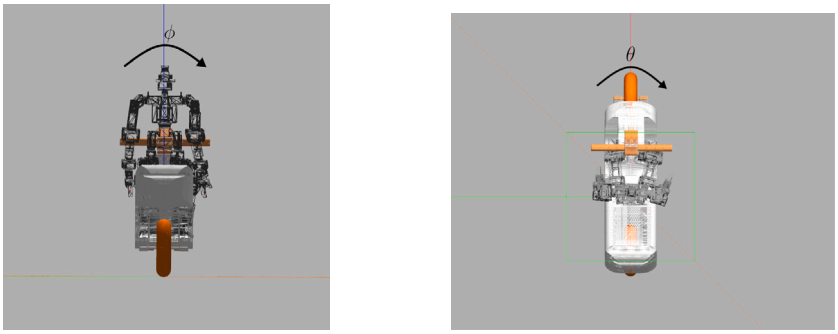


Fig. 8. Robot's system diagram.



(a) Rear view of the robot-scooter model. The tilt angle is represented by ϕ.

(b) The top view of the robot-scooter model. The yaw/orientation angle is represented by θ.

Fig. 9. Views of the resulting 3D model of the robot-scooter system designed with the CAD software.

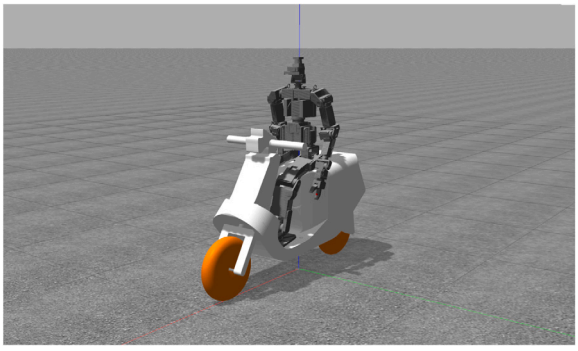


Fig. 10. 3D model of the robot-scooter system in the simulator.

2. Tracking a straight line with increasing disturbance forces.

Several PID gains are explored. In order to obtain a good performance, different gains are utilized for each velocity but kept the same for the two experiments enumerated above. The gains are presented in Table 6 in the Results and Discussion section.

4.2. PPO - Proximal Policy Optimization

Two of the most common problems that plague DRL methods are data efficiency and learning robustness. Data efficiency, also called sample complexity, is the capacity of a method to learn for each input unit. If a method is data-efficient it requires less data (and thus less training time) to learn good policies. The method should also be able to extrapolate and generalize reasonably to similar but not the same type of data. The other problem DRL methods commonly face is loss of progress. Because the agents are interacting with the environment

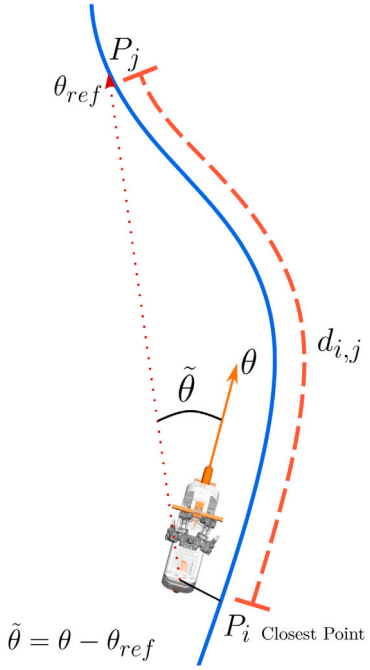


Fig. 11. Path tracking is done by computing the orientation error in relation to a point 7 m along the path. The path to follow is represented by the contiguous blue line, and the red line represents the distance between the points that make up the path.

and generating the training data at the same time, the training signal is noisy. Methods based on policy gradient (as well as others) can suffer from loss of progress due to this. To improve on these factors, methods based on trust regions perform limited updates, in order to avoid changing the distribution too much. TRPO (Schulman et al., 2015) is an example of such a method, but its formulation is somewhat complicated and does not support neural net architectures that use dropout or parameter sharing (Schulman et al., 2017).

The most recent and state-of-the-art method that shows improvements over it is called PPO (Schulman et al., 2017). The PPO has shown outstanding performance at continuous control tasks used for many application domains in robotics (Zhang et al., 2022). In this study, our proposed DRL algorithm's main structure is based on PPO as described in the following. Trust region methods make use of a probability ratio between the new and old actions probability distribution defined as shown in Eq. (4).

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \tag{4}$$

where, π_θ indicates the policy, and θ its defines its parameters. a_t and s_t are the action and state at timestep t.

In addition to simpler formulation compared to TRPO, PPO empirically shows better results in continuous control tasks (Schulman et al., 2017). From the original paper (Schulman et al., 2017), a surrogate loss function is defined as the following.

$$L^{CLIP}(\theta) = \hat{E}[\min(r_t(\theta))\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t] \tag{5}$$

where, $\hat{A}_t$ is advantage function estimator and e is a hyperparameter that determines the amount of clipping. The $\hat{E}$ denotes the average over a batch of data. In architectures where the model shares parameters between the policy and value function estimators, it is necessary to add another term to Eq. (5), referring to the value function error as shown in Eq. (7).

$$L_t^{VF} = (V_\theta(s_t) - V_t^{\text{T}})^2 \tag{6}$$

where the first term is the prediction of the value function estimator for state s_t and V_t^T as the target value. The resulting objective function combines the surrogate loss with the value function error in Eq. (7).

$$L_t^{CLIP+VF}(\theta) = \hat{E}_t[L_t^{CLIP}(\theta) - c_k L_t^{VF}(\theta)] \tag{7}$$

where c_k is a hyperparameter for modifying the magnitude of the value function error.

In PPO the training loop is done by alternating between sampling from multiple agents running in parallel and optimization steps. At each iteration, N agents collect a batch of data for T timesteps. Then, the loss from Eq. (7) is computed and optimized for a number of epochs using algorithms such as SGD, Adam, or similar algorithms (Ruder, 2016). The algorithm for training, from (Schulman et al., 2017), is presented as shown in Algorithm 1.

Algorithm 1 Proximal Policy Optimization Training Loop

```
for Iteration i ∈ {1, 2, ... } do
  for Agent k ∈ {1, ..., N} do
    Run policy π_θold for T timesteps
    Compute advantage estimates Â_1, ..., Â_T
  end for
  Optimize loss L_t^CLIP+VF for K epochs and batch size M ≤ NT
end for
```

As demonstrated in Algorithm 1, in our study, the PPO training loop plays a crucial role in training our DRL agent to balance all the control and path tracking of the robot-scooter system. The algorithm iteratively trains the agent, represented by the variable k, by running the policy $\pi_{\theta_{\text{old}}}$ for a specified number of timesteps T and computing advantage estimates $\hat{A}_1$ to $\hat{A}_T$. The advantage estimates capture the expected improvement in performance by taking certain actions in a given state. Subsequently, the algorithm optimizes the loss function $L_t^{CLIP+VF}$ using a combination of clipped policy objective and value function terms, which ensures stability during training. This training loop, executed for multiple iterations denoted by i, allows our agents to learn effective balance control and path tracking policies through reinforcement learning, ultimately enhancing the overall performance of the robot-scooter system.

4.3. Our DRL agent based on PPO

In this paper, the training and subsequent testing of the agent are performed using Isaac Gym, a recently released simulator from NVIDIA with GPU-based training capabilities. Makoviychuk et al. (2021). Issac Gym is used in different robotics research due to its high-performance learning platform on GPU benefits (Le et al., 2022). Fig. 12 shows the 3D model loaded in the simulator.

The problem of long compute times required for training DRL agents is addressed by Isaac Gym, which uses NVIDIA PhysX engine (Makoviychuk et al., 2021; NVIDIA, 2022). This physics engine provides access to GPU tensors via a PyTorch wrapper, which by using a single GPU, it is possible to run multiple environments in parallel, and training is orders of magnitude faster than on a CPU.

In this paper, two distinct agents are trained: one for balance control, which attempts to keep the robot-scooter system upright, and a second agent which is capable of receiving turning rate commands, used for path tracking. Table 3 presents the experiments setup and computational system used for the training. Both agents are trained with 64 parallel environments, shown in Fig. 13. The same neural network architecture is used for both agents: a three-layer MLP network with 256, 128, and 64 neurons, respectively. The activation function utilized as scaled exponential linear units (SELUs) (Klambauer et al., 2017). Table 4 presents the relevant PPO's hyperparameters that are utilized.

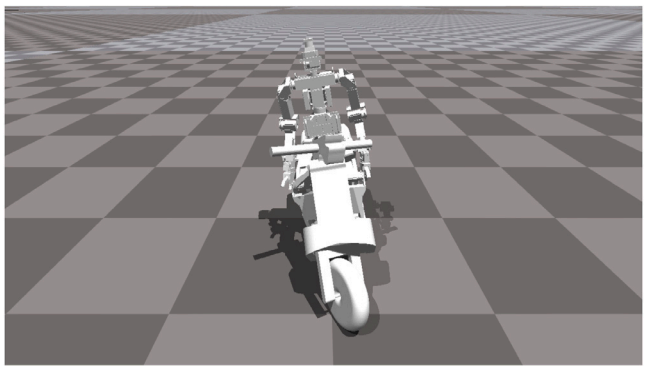


Fig. 12. 3D model of the robot-scooter system in NVIDIA's Isaac Gym simulator. Isaac gym has been utilized for training and evaluating the DRL agents.

Table 3
Relevant specifications of the computer used to train the DRL agents.

GPU	1x NVIDIA RTX 2080 Super (8 GB)
CPU	8-Core AMD Ryzen 7 3700X
RAM	32 GB

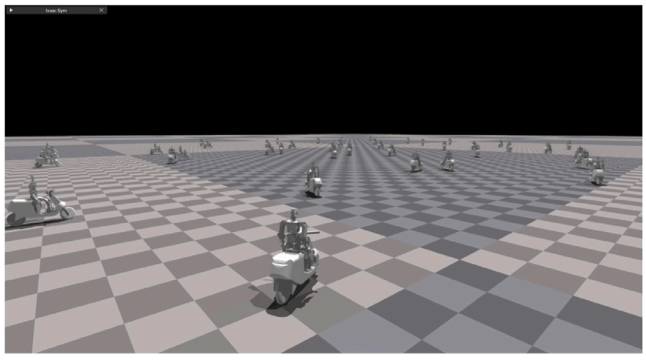


Fig. 13. By using GPU tensors it is possible to train several agents in parallel in the Isaac Gym simulator.

Table 4
Hyperparameters used for training of the PPO agents.

Clip range (ϵ)	0.2
Sampling steps (T)	10
Optimization steps	10
Minibatches (per environment)	2
Optimizer	Adam
Learning rate	3.0e−5
Gamma (γ)	0.99

There are 8 observations indicating the state of the environment as input to the model, which indicates states of the environment:

- 1 value for the steering angle
- 1 value for the velocity of the rear wheel
- 3 values for the orientation
- 3 values for the angular velocities

The action space is indicated by the final output of IK as steering. So robot sets a new angle for the steering as an action. To be able to act robustly for any velocity the agent has set to a random velocity in the range $[10, 40]$ rad/s every time the environment gets started.

In the following sections, the implementation details and experiments conducted for the DRL agents are presented separately in detail.

4.3.1. Balance control

The goal of the balance control is to keep the robot-scooter system perfectly upright, that is, maintaining a $\phi = 0$. The reward function is formulated as $R = 1.0 - \phi^2$. This means that the agent is given a maximum reward of 1.0 when $\phi = 0$ and squarely less reward the farther away from the intended tilt it is. In addition, the agent receives a negative reward of −2 for tipping the robot-scooter system, i.e. when $\text{abs}(\phi) > 65°$.

By using a simulator it is possible to directly access the internal state of any object being simulated. To keep the agent somewhat grounded in reality the only observations that are fed to the model are sensor data that would be available for the real robot-scooter system.

Training has been performed for a total of 4 million timesteps, which correspond to approximately two hours. The following two experiments are designed to analyze the behavior of the balance control agent:

1. Balance under no disturbances.
2. Balance with lateral disturbances of increasing magnitude.

The disturbance forces are generated by hurling a box at a constant velocity at the center of mass of the agent. The difference in velocity of the box is used to compute the resulting force using the classic physics equation $F = m \cdot \Delta V$. Increasing the velocity resulted in forces of a larger magnitude.

4.3.2. Commanded control – Path tracking

In order to track it is necessary to be able to turn the robot-scooter system. An agent that is capable of receiving $\dot{\theta}$ commands has been trained. The intended turning rate $\dot{\theta}^{\text{targ}}$ is added as another observation (input) to the model. The three components of linear velocity are also included, for a total of 12 observations in this version of the agent. All of the observations are grounded in reality and are data that could be obtained from sensors such as an IMU. The reward function is defined in (8).

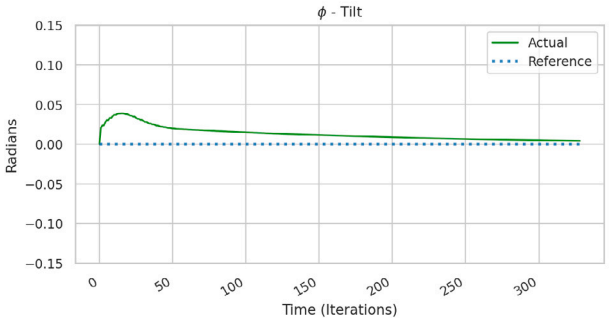
$$R = 1.0 - |\dot{\theta} - \dot{\theta}^{\text{targ}}|^{\frac{1}{5}}. \tag{8}$$

Whenever the agent can perfectly match the turning velocity to the target velocity, it will get a maximum reward of 1.0, and increasingly less reward the farther away it moves from the goal velocity. The fraction term makes the region where the reward is given relatively small, which prevents the agent from taking "lazy" actions and converging into unintended behavior.

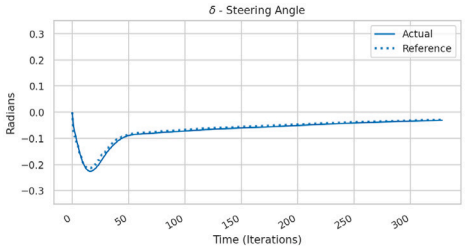
In the same manner as the previously described agent, this one is also set to random velocity in the range $[10, 40]$ rad/s when the environment restarts. To provide the agent with rich samples from a variety of $\dot{\theta}$ commands, every 2.5 s, an offset is sampled from a uniform distribution in the range $[-5.7°, 5.7°]$, which corresponds to 0.1 rad/s. With these configurations, the agent has enough time to perform actions and try to maximize the reward. At the same time, the command updates are frequent enough that the agent has a rich variety of samples. This agent is trained for a total of 6.1 million timesteps, which corresponds to about 2.5 h. After the agent is trained, it is capable of receiving turning commands at any speed. The path is tracked by using the method as the PID controller, explained in Section 4.1.2. This DRL agent's control output is the turning rate instead of the tilt angle. A single proportional term with a gain of 1.0 is used to set the target turning rate according to the orientation error.

Two tests are conducted to analyze the capacity of the DRL agent to track the paths where the disturbances are realized in the same manner as detailed in the previous section:

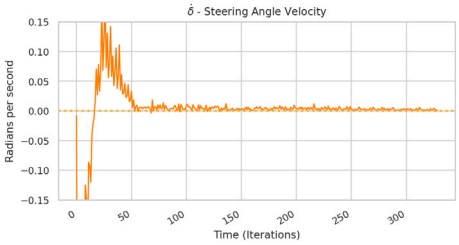
1. Tracking a straight line path and a sinusoidal path under no disturbances;
2. Tracking a straight line path while under increasing disturbances.



(a) Reference and actual tilt angle.

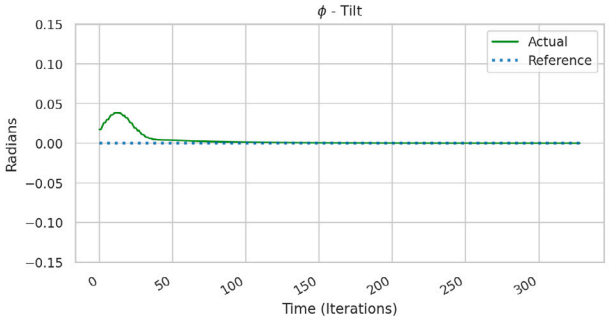


(b) Reference and actual steering angle.

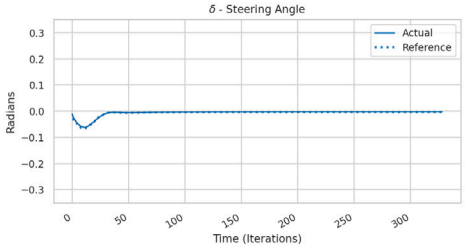


(c) Rate of change of steering angle.

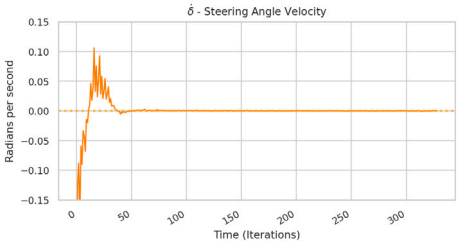
Fig. 14. Balance control with a reference tilt angle of 0° using PID controller and no disturbances at a velocity of 10 rad/s (2.5 km/h).



(a) Reference and actual tilt angle.



(b) Reference and actual steering angle.



(c) Rate of change of steering angle.

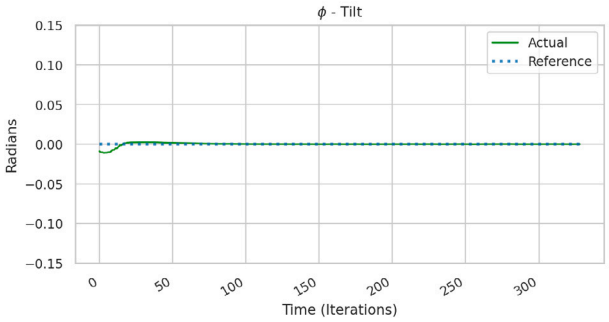
Fig. 15. Balance control with a reference tilt angle of 0° using PID controller and no disturbances at a velocity of 20 rad/s (5 km/h).

5. Results and discussion

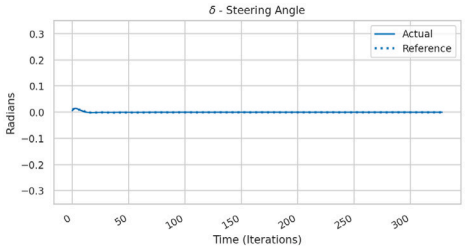
This section presents relevant data and extensive experimental results collected from the proposed experiments. The experiments demonstrate two control methods (PID and proposed DRL) in separate subsections. The experiments generally applied to two scenarios: balance of the scooter-robot system and tracking the paths. This resulted in four main sets of experiments, including relevant metrics that are measured and parameter values. Additionally, a separate section provides further discussion for the comparison of the PID and DRL algorithms. The main part of simulations and implementations have been published in the GitHub link.[1]

All the theoretical results in the section address the challenge of balancing a two-wheeled scooter using a humanoid robot. The practical motivation is to develop versatile robots capable of acquiring various

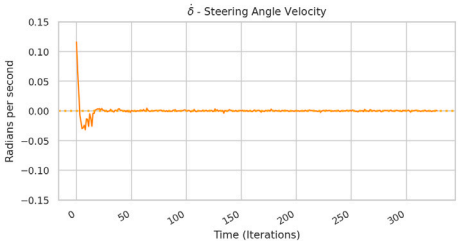
[1] https://github.com/guichristmann/thormang3-gogoro-PPO.



(a) Reference and actual tilt angle.

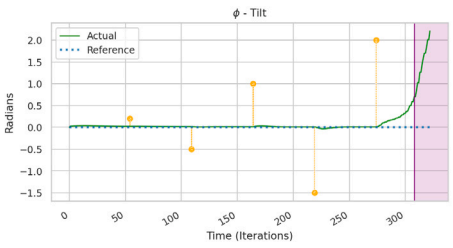


(b) Reference and actual steering angle.

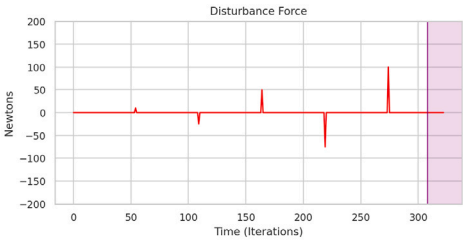


(c) Rate of change of the steering angle.

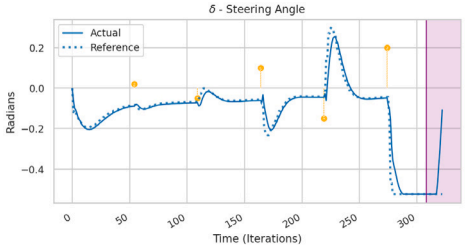
Fig. 16. Balance control with a reference tilt angle of 0° using PID controller and no disturbances at a velocity of 40 rad/s (10 km/h).



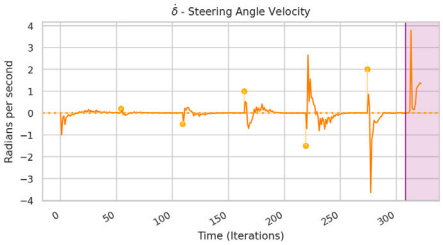
(a) Reference and actual tilt angle.



(b) Lateral disturbance force.



(c) Reference and actual steering angle.



(d) Rate of change of the steering angle.

Fig. 17. At a velocity of 10 rad/s (2.5 km/h) the largest lateral force the system can withstand before tipping over is 100 N. The purple region indicates that the scooter has fallen over. The vertical lines indicate the iterations that a disturbance force has been applied.

skills. In this paper, we propose a DRL algorithm for steering control, compared to a classical PID controller, which highlights the potential of DRL algorithms in solving complex control tasks and enabling robots to adapt to different environments.

5.1. Balance control – PID controller

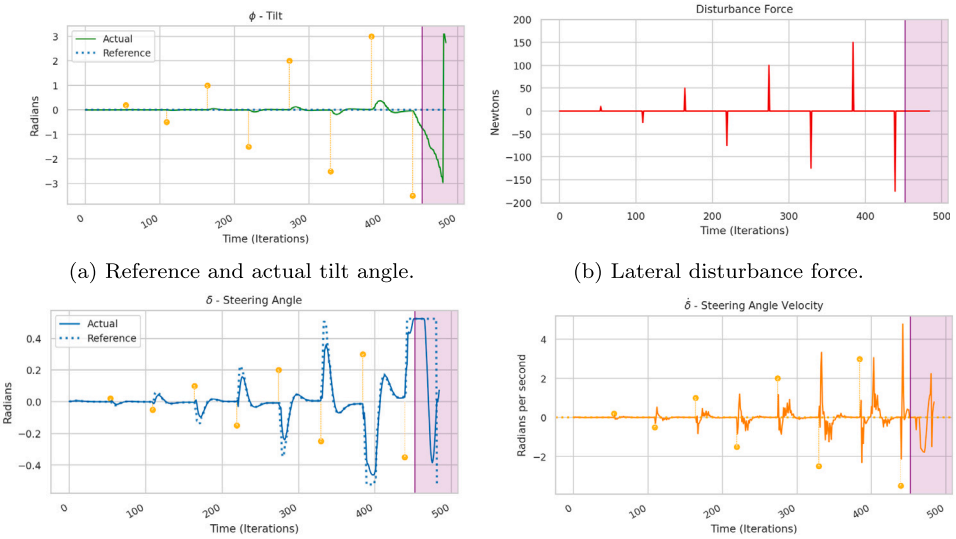
The goal of this experiment set is to analyze the balancing capability of the robot-scooter system using PID control. Balance is defined as tracking a reference tilt angle (i.e. 0° for completely upright as reference) in a stable manner. The experiment is realized for the three sets of velocities: 2.5 km/h, 5 km/h, and 10 km/h.

Table 5
PID parameters for balance control.

System velocity	P	I	D
10 rad/s (2.5 km/h)	8.0	0.04	4.0
20 rad/s (5 km/h)	2.0	0.04	4.0
40 rad/s (10 km/h)	1.0	0.04	2.0

5.1.1. PID controller - Upright balance

Utilizing the parameters from Table 5 the behavior of the controller is investigated by tracking a tilt reference of 0°, i.e. keeping the robot-scooter system perfectly upright. Figs. 14(a), 15(a) and 16(a) show that, for the three velocities, the system is able to reach a steady state of balance. Notably, the higher the system velocity the easier it is to



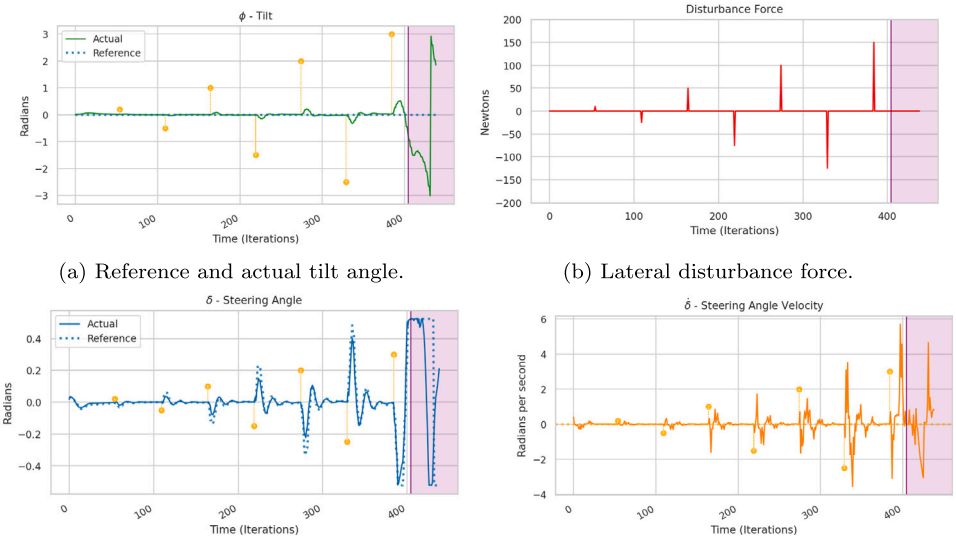
(a) Reference and actual tilt angle.

(b) Lateral disturbance force.

(c) Reference and actual steering angle.

(d) Rate of change of the steering angle.

Fig. 18. At a velocity of 20 rad/s (5 km/h) the largest lateral force the system can withstand before tipping over is 175 N. The purple region indicates that the scooter has fallen over. The vertical lines indicates the iterations that a disturbance force has been applied.



(a) Reference and actual tilt angle.

(b) Lateral disturbance force.

(c) Reference and actual steering angle.

(d) Rate of change of the steering angle.

Fig. 19. At a velocity of 40 rad/s (10 km/h) the largest lateral force the system can withstand before tipping over is 150 N. The purple region indicates that the scooter has fallen over. The vertical lines indicate the iterations that a disturbance force has been applied.

reach the stable state, as evidenced by the earlier convergence to the reference signal and smaller PID gains at high velocities. Additional evidence can be observed in Figs. 14(c), 15(c), and 16(c), which show that the control is much easier on the steering actuator, with reduced rates of change. These observations are in line with the literature (and common sense): it is much harder to balance (and control) a TWD at low velocities (Sikander and Prasad, 2019).

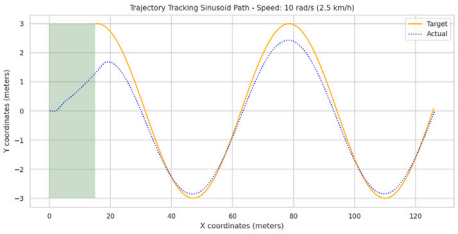
5.1.2. PID controller - Upright balance under disturbances

The same set of PID parameters presented from Table 5 in Section 5.1.1 has been utilized in this experiment. Figs. 17, 18, and 19 shows the system's response after being disturbed by forces of increasing magnitude until tipping over. At the lowest velocity, the largest force the system could recover is just 100 N, as shown in Fig. 17(b). At higher velocities, it could withstand disturbance forces of 175 N and 150 N and still recover successfully, as shown in Figs. 18(b) and 19(b), respectively. From these observations, we can infer that the system is more robust to perturbations at higher speeds.

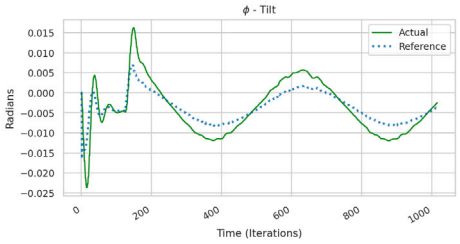
5.1.3. PID controller - Path tracking

The goal of this experiment set is to analyze the capacity of the robot-scooter system to track a desired path using a PID controller. The controller consisted of two PID controllers: 1. manipulate the steering angle to maintain and desired tilt angle, and 2. manipulate the desired tilt angle according to the path error. The following sections present the relevant data for judging the control behavior of this controller at the three sets of velocities, tracking two types of the path: a sinusoidal and a straight-line path. The parameters and gains utilized for each experiment are presented as well.

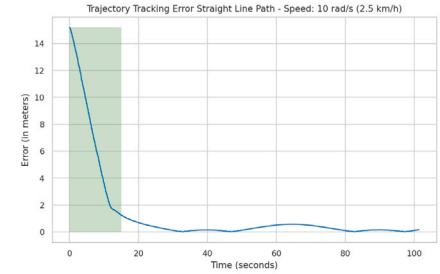
Table 6 contains the PID gains utilized for the two controllers in the path-tracking experiments in which the gains from the balance control (See Section 5.1) are kept. However, at the highest velocity, the system



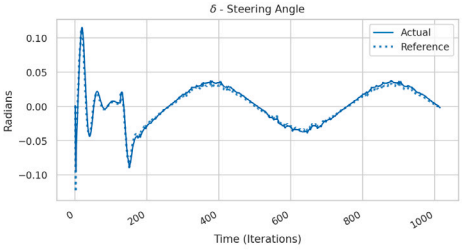
(a) Target and actual path. Green indicates approaching region.



(b) Reference and actual tilt angle.

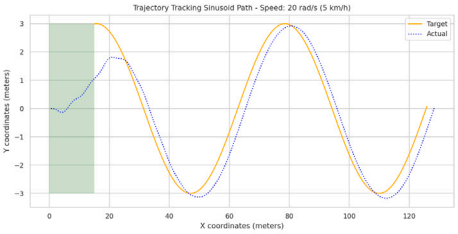


(c) Path error. Green indicates approaching region.

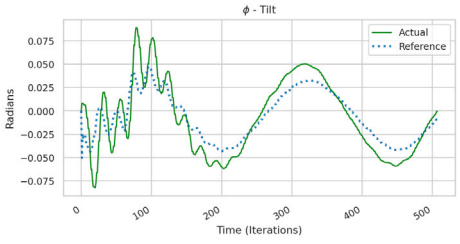


(d) Reference and actual steering angle.

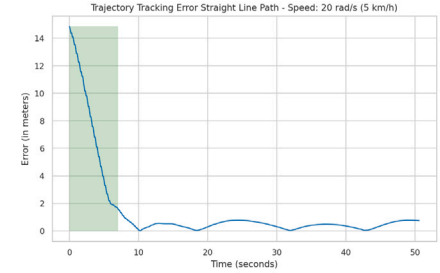
Fig. 20. Tracking the sinusoidal path at a velocity of 10 rad/s (2.5 km/h). The average error is 0.3186 m.



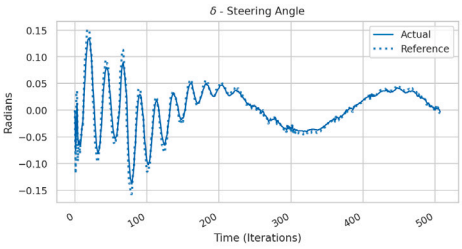
(a) Target and actual path. Green indicates approaching region.



(b) Reference and actual tilt angle.



(c) Path error. Green indicates approaching region.



(d) Reference and actual steering angle.

Fig. 21. Tracking the sinusoidal path at a velocity of 20 rad/s (5 km/h). The average error is 0.4853 m.

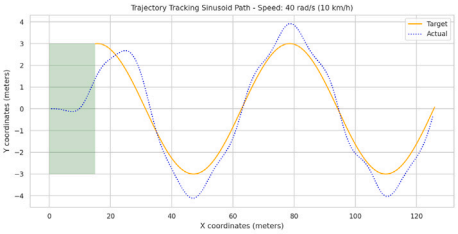
Table 6
PID parameters for the two PID controllers in the path tracking experiment.

Mode	Velocity = 10 rad/s	Velocity = 20 rad/s	Velocity = 40 rad/s
Balance	P = 8.0, D = 4.00, I = 0.040	P = 2.0, D = 4.00, I = 0.040	P = 0.6, D = 0.8, I = 0.0
Tilt reference	P = 0.1, D = 0.05, I = 0.001	P = 0.2, D = 0.25, I = 0.001	P = 0.8, D = 0.2, I = 0.0

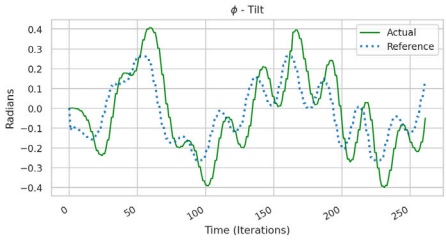
is unstable when tracking the paths, and the gains are modified. During this set of experiments, it is found that the controller is very sensitive to the combination of the PID values, especially at the highest velocity. The gains are settled so that the controller could reasonably track both of the proposed paths in a stable manner.

5.2. PID controller - Tracking sinusoidal path

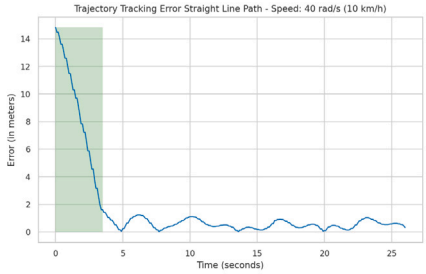
The proposed controller is able to track the sinusoidal path at all speeds, as shown in Figs. 20, 21, and 22. The green rectangles on the graphs indicate the regions where the robot-scooter system is approaching the path, and are discarded when computing the average



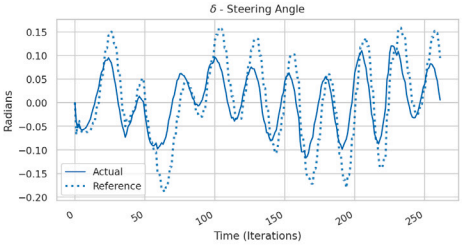
(a) Target and actual path. Green indicates approaching region.



(b) Reference and actual tilt angle.

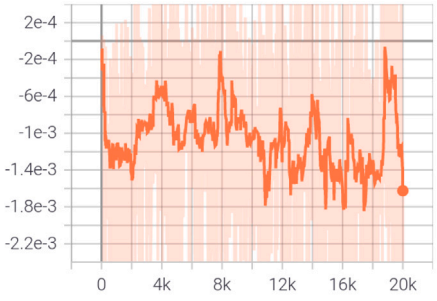


(c) Path error. Green indicates approaching region.

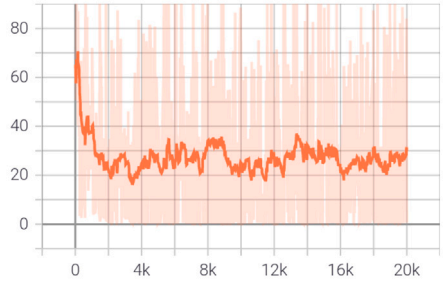


(d) Reference and actual steering angle.

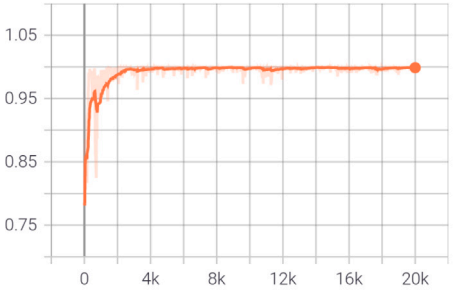
Fig. 22. Tracking a sinusoidal path at a velocity of 40 rad/s (10 km/h). The average error is 0.5679 m.



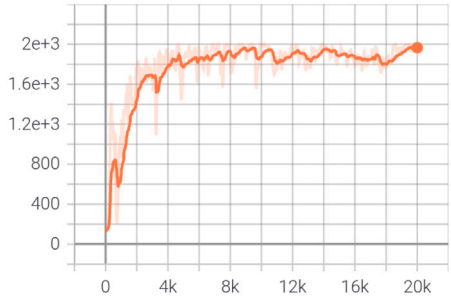
(a) PPO's surrogate loss over 20k training iterations.



(b) The value function loss over 20k training iterations.



(c) The mean reward per step increases and converges close to the maximum reward value (1.0) as training progresses.



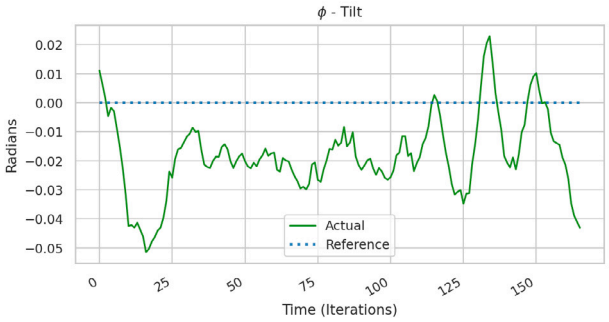
(d) The mean episode length increases and converges to the maximum episode length (2000) as training progress.

Fig. 23. Training graphs of the PPO model for balancing the robot-scooter system (the agent) upright. The agent controls the steering joint actuator and is given more reward (maximum of 1.0) the closer the tilt angle is to 0.

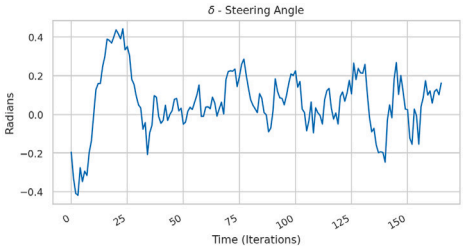
error. The best performance is clearly at the lowest velocity. In addition to having the smallest average error at just 0.3186 m, it also had the smallest oscillations and is the most stable controller. At other velocities, the controller is still able to generally track the sinusoidal path but suffers from instability and oscillations. This is especially clear at the highest velocity, in Fig. 22(a), which also had the largest path error.

5.3. DRL - Balance control

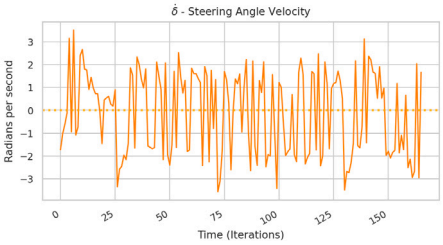
The goal of this experiment set is to analyze the balancing capability of the robot-scooter system using the trained DRL (PPO) control. The DRL agent is rewarded for keeping the robot-scooter system perfectly upright, i.e. a tilt reference angle of 0° ($\phi_{ref} = 0$). The agent is trained with velocities in the range of 10 rad/s to 40 rad/s, but, for the purpose of comparing the results with the PID controller from Section 5.1, the



(a) Reference and actual tilt angle.

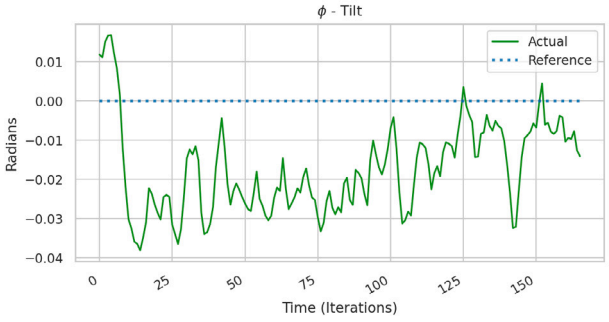


(b) Reference and actual steering angle.

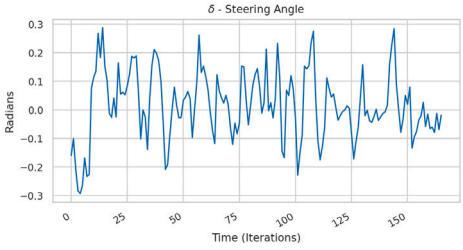


(c) Rate of change of steering angle.

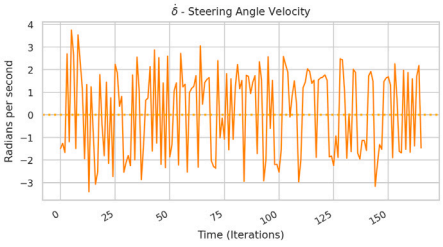
Fig. 24. Learned balance control with a reference tilt angle of 0° of the DRL agent under no disturbances at a velocity of 10 rad/s (2.5 km/h).



(a) Reference and actual tilt angle.



(b) Reference and actual steering angle.



(c) Rate of change of steering angle.

Fig. 25. Learned balance control with a reference tilt angle of 0° of the DRL agent under no disturbances at a velocity of 20 rad/s (5 km/h).

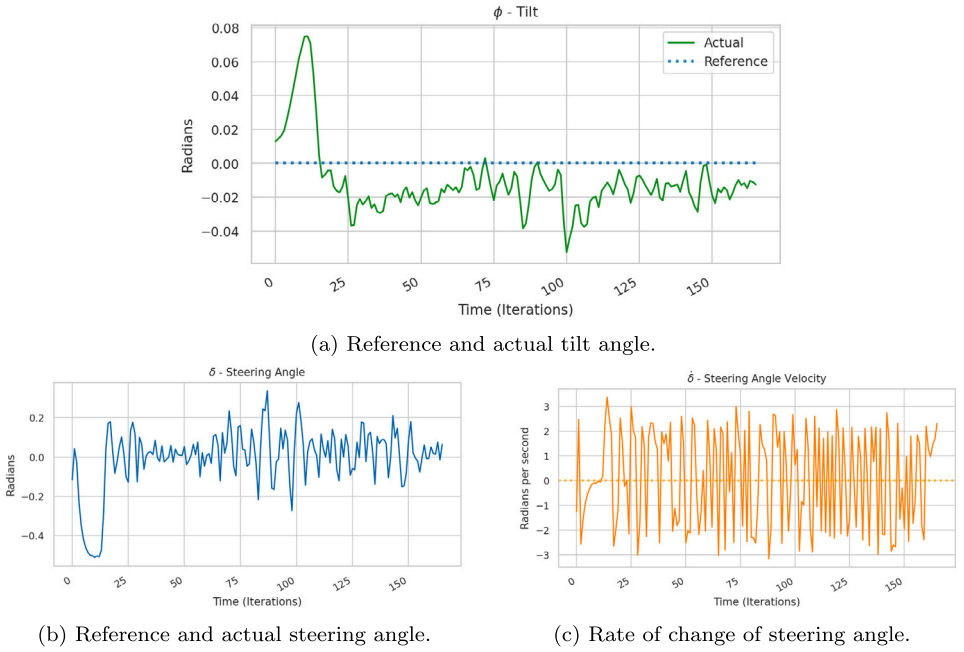
results are presented for the same three sets of velocities: 10 rad/s (2.5 km/h), 20 rad/s (5 km/h), and 40 rad/s (10 km/h).

The training progression graphs are presented in Fig. 23. We can observe that the training is very stable and the progress is robustly maintained. This is one of the main advantages of the PPO algorithm over previous continuous control DRL algorithms such as DDPG (Lillicrap et al., 2015), which can suffer from progressive loss during training. The PPO-based system is trained using 4,000,000 episodes. Figs. 23(c) and 23(d) show that the agent quickly converges and robustly maintains its performance. The maximum reward is 1.0 and the maximum episode length is 2000. The following Sections 5.4 and 5.4.1 all use this same trained model and present its response at different velocities and disturbance forces.

5.4. DRL - Upright balance

An agent that could balance the system upright at velocities ranging from 10 rad/s to 40 rad/s has been successfully trained. Observing Figs. 24, 25, and 26 we can see that the agent learned to manipulate the steering joint to track the reference tilt angle of 0°, and converged to have a small error of approximately 2° off the reference.

Probably due to the fact that the agent is given no limitations on the amount of input (actions) it can perform, it ended up being very hard on the steering joint actuator, as shown in Figs. 24(c), 25(c), 26(c). This factor seems to be unaffected by the velocity of the system. Even at the maximum tested velocity, where the system is more stable and easier to control, the agent still performs rates of change up to 180°.

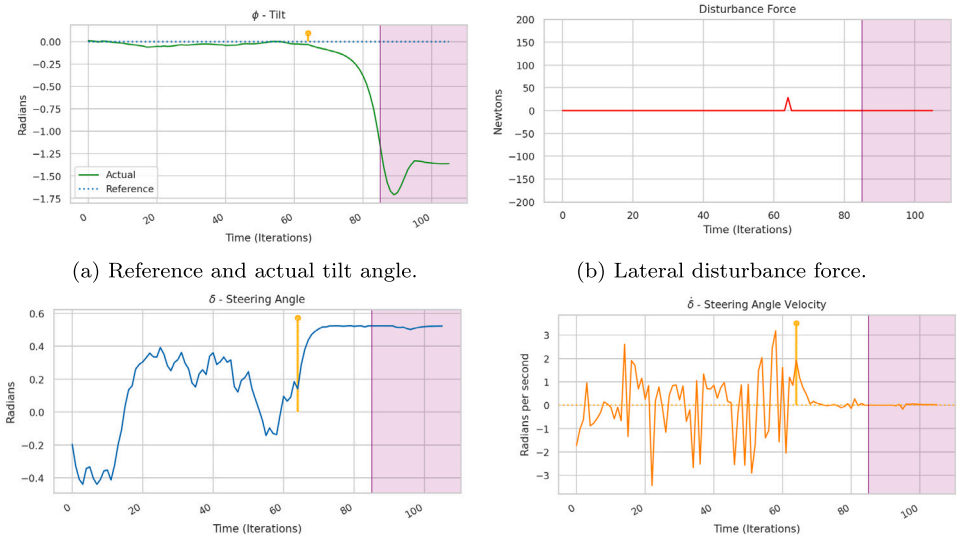


(a) Reference and actual tilt angle.

(b) Reference and actual steering angle.

(c) Rate of change of steering angle.

Fig. 26. Learned balance control with a reference tilt angle of 0° of the DRL agent under no disturbances at a velocity of 40 rad/s (10 km/h).



(a) Reference and actual tilt angle.

(b) Lateral disturbance force.

(c) Reference and actual steering angle.

(d) Rate of change of the steering angle.

Fig. 27. At a velocity of 10 rad/s (2.5 km/h) the DRL agent is especially sensitive to disturbances and tips over with a force of less than 50 N. The purple region indicates that the scooter has fallen over. The vertical lines indicate the iterations that a disturbance force has been applied.

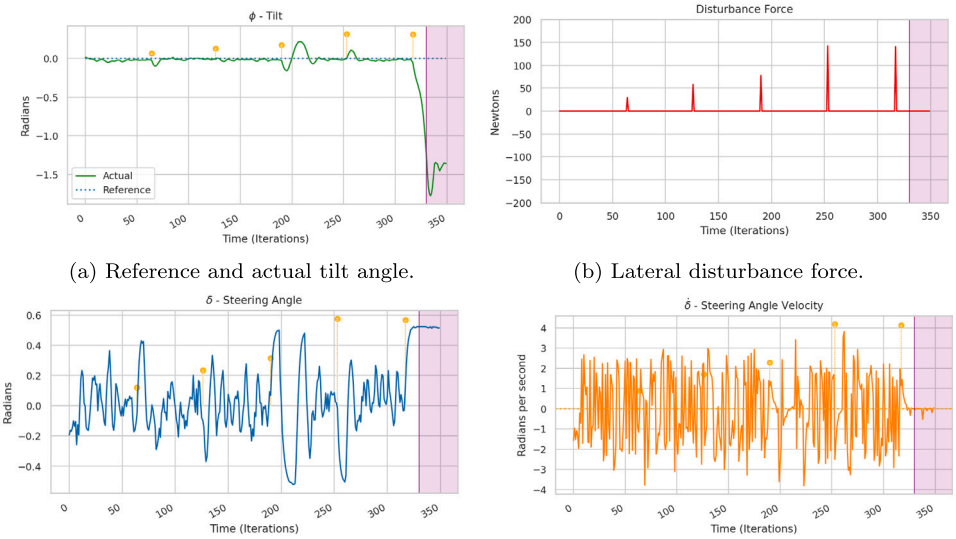
5.4.1. DRL - Upright balance under disturbances

Even though the agent is not subjected to disturbance forces during training it still developed the ability to withstand such perturbations to an extent. Figs. 28 and 29 show that the agent is capable of withstanding perturbations up to 150 and 175 N, at the velocities of 20 rad/s (5 km/h) and 40 rad/s (10 km/h), respectively. However, at the low velocity of 10 rad/s (2.5 km/h), even a perturbation force as small as 50 N can cause the system to fall irrecoverably. As shown in Fig. 27, and more specifically Fig. 27(c), as soon as the system is subject to the perturbation the agent moved the steering angle to its limit, but still could not recover from the falling motion.

An interesting occurrence that we can observe is that even the lower disturbance forces induced oscillations in the tilt in some instances. Notably, in Fig. 28(a) it is shown that a force of fewer than 50 N caused such oscillations, even though larger forces did not provoke such an effect. One possible explanation is that since the agent actuates with such a large magnitude and frequency, it could be more sensitive to forces depending on the current cycle or internal state. In addition, one would expect that if the agent is subject to such perturbations during training this effect should disappear or become greatly minimized.

5.4.2. DRL - Path tracking

In order to perform the path tracking an agent that could receive target turning rate commands is trained. In Fig. 30 it is possible to observe the training graphs and the progress of the training. The agent has been trained at velocities ranging from 10 rad/s up to 40 rad/s. During training, the agent is rewarded for reaching a commanded turning rate, with a maximum reward value of 1.0. After training is finished, the agent could receive and execute turning commands up to



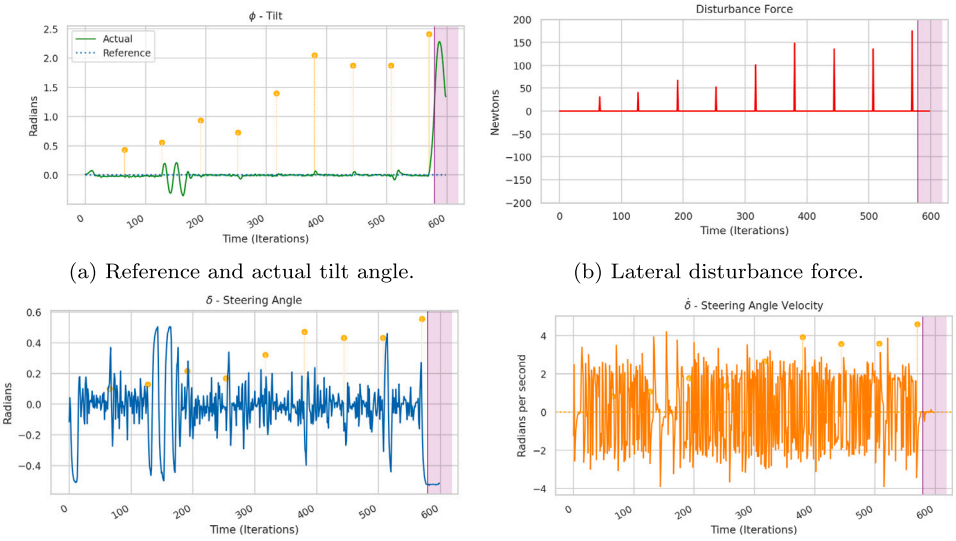
(a) Reference and actual tilt angle.

(b) Lateral disturbance force.

(c) Reference and actual steering angle.

(d) Rate of change of the steering angle.

Fig. 28. At a velocity of 20 rad/s (5 km/h) the DRL agent can sometimes withstand forces of approximately 150 N, but will generally fall. The purple region indicates that the scooter has fallen over. The vertical lines indicate the iterations that a disturbance force has been applied.



(a) Reference and actual tilt angle.

(b) Lateral disturbance force.

(c) Reference and actual steering angle.

(d) Rate of change of the steering angle.

Fig. 29. At a velocity of 40 rad/s (10 km/h) the DRL agent can reliably withstand forces of 150 N but falls with larger forces. The purple region indicates that the scooter has fallen over. The vertical lines indicate the iterations that a disturbance force has been applied.

40° per second in either direction. For all of the experiments below the same model is utilized.

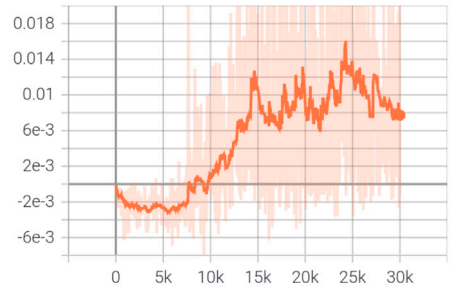
The agent learned to reliably turn the robot-scooter system at the commanded turning rate. For low velocities and large commands, the agent learned an emergent behavior where he would execute a smaller turning rate in the same direction to avoid falling over. Additionally, the turning rate signal (θ) from the simulator is very noisy, and the agent converged to match the command on average. At first sight, by looking at the mean reward per training step in Fig. 30(c), one might think that the agent did not learn a robust behavior. Taking into account the previous factors, it becomes more clear that agents indeed learn useful behaviors, and it is indeed maximizing the reward given the limitations (e.g. falling at low velocities if trying to turn too much).

The next sections show further evidence of the agent's learned behaviors. In these sections, the tilt graph is replaced with the current and intended turning rate ($\dot{\theta}$) since it is the command variable of the DRL agent.

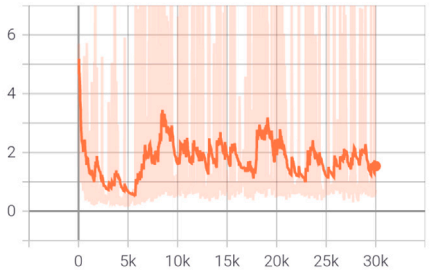
5.5. Comparison - PID controller and DRL

The PID controller is one of the most commonly used controllers, which is a type of feedback controller that regulates and stabilizes a system. The PID controller has successfully been used for balance control tasks in different applications, Joseph et al. (2022). In this paper, we propose scooter balance control by a general-purpose humanoid robot in the presence of external disturbances for the first time. To demonstrate the proposed algorithm's performance compared to the most popular existing control approach, this section summarizes and presents balance experiments on the most complex case sinusoidal path tracking control for both the PID controller and the DRL. In Fig. 31, we compared the average tracking error comparison between the PID controller and the DRL agent for the Sinusoidal path.

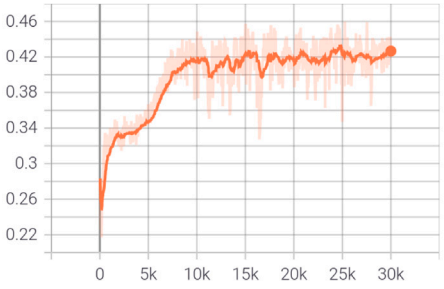
To further illustrate the effectiveness of the proposed algorithm, we provide a detailed analysis and discussion comparing the performance



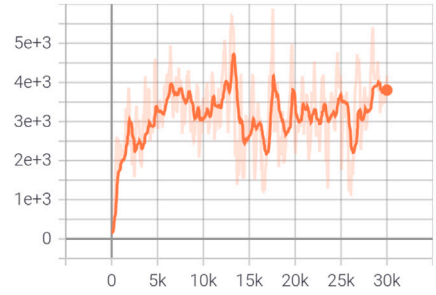
(a) PPO's surrogate loss over 30k training iterations.



(b) The value function loss over 30k training iterations.



(c) The mean reward per step increases as training progresses.



(d) The mean episode length increases as training progress.

Fig. 30. Training graphs of the PPO model for the commanded agent. The agent is trained by being rewarded for matching a command for an intended turning rate and punished by tipping over.

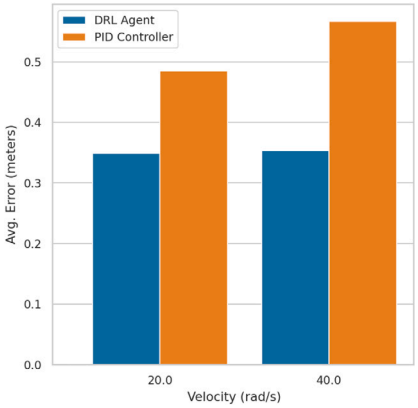


Fig. 31. Average tracking error comparison between the PID controller and the DRL agent on the path tracking.

of the PID controller and the DRL agent. Specifically, we focus on the example of sinusoidal path tracking control, which represents a challenging and complex scenario for balancing a two-wheeled scooter. Fig. 31 presents the comparison of the average tracking error between the PID controller and the DRL agent for the Sinusoidal path. The results clearly demonstrate the superior performance of the DRL agent compared to the PID controller in different average speeds (52% average improvement). Although the PID controller generally has an easier development process, it still suffered from oscillations and overshooting problems. The gains are especially sensitive at the highest velocity, and the joint PID controller system is not robust. One of the main advantages of the DRL agent is that a single model can perform well for different speeds and turning commands, while the PID controller needed different gains for each velocity and is generally not as stable even after tuning.

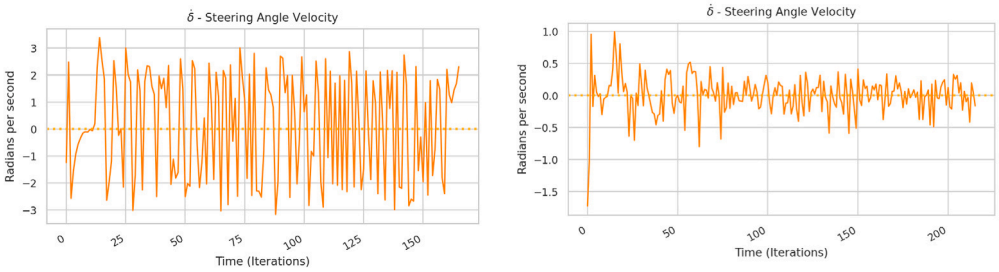
An interesting result can be seen regarding the steering velocities when comparing the DRL agent that performed balance control and the agent that performed turning commands. As seen in Fig. 32, the steering velocities and frequencies are much smaller for the commanded DRL agent. This implies that the commanded agent is more feasible and has more potential for application to the real robot and scooter. One possible explanation for this phenomenon is that the DRL agent that performs only upright balancing learned to exploit high steering angles to keep the reference as close to 0.0 as possible. Because the agent is not punished for high-frequency and high-magnitude steering actions, this hyper-optimization of the problem results. On the other hand, the commanded agent had to tackle multiple turning rates, which resulted in a more stable behavior.

5.6. Real robot steering velocity test

In order to evaluate the feasibility of the controller commands, a steering velocity test is realized using the real robot and scooter. Fig. 7 shows the sequence of frames of the robot gripping the steering bar. The steering bar has been rotated to its limits using inverse kinematics at the highest velocity possible. The maximum measured velocity is 1.5 rad/s; the resulting data is shown in Fig. 33. Comparing this result with the best-case scenario for the PID controller and DRL agent (both at the velocity of 40 rad/s) shown in Fig. 34, one could say that the robot is capable of realizing these commands to a certain extent. However, in addition to the magnitude, the frequency is also a major problem and inverse kinematics is not fast enough to perform these action rates.

5.7. Discussion

It is important to highlight the stability and robustness advantages offered by the DRL agent. While the PID controller requires meticulous



(a) Steering velocity for balance-only DRL agent. Steering is performed at high-magnitude and high-frequency.

(b) Steering velocity for commanded DRL agent. Steering is still at a high-frequency but much lower magnitude.

Fig. 32. The commanded DRL agent has a more desired behavior: lower-magnitude steering commands.

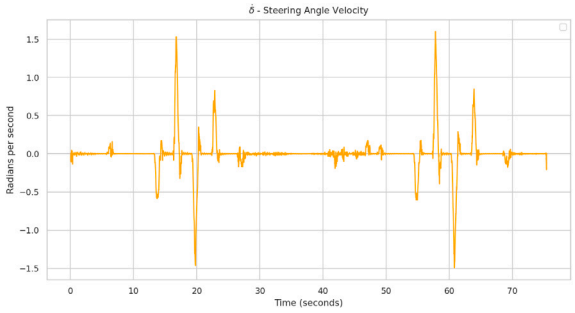


Fig. 33. Steering velocities of the real robot. The steering joint of the real scooter is turned at maximum velocity using inverse kinematics. The maximum measured velocity is 1.5 rad/s.

gain tuning for each velocity, the DRL agent demonstrates excellent stability across different speeds and turning commands without the need for parameter adjustments. This inherent adaptability of the DRL agent makes it more reliable and suitable for real-world applications.

By looking at the results in this paper, it is clear that the DRL agent exhibits remarkable generalizability, allowing it to perform well in various control scenarios. Unlike the PID controller, which relies on specific gains for each velocity, the DRL agent can adapt its steering control to different speeds and turning commands, resulting in improved balance and path tracking. This versatility further enhances the practicality and usability of the proposed algorithm. In summary, the Sinusoidal path tracking control's comparative analysis clearly demonstrates the results' effectiveness. The DRL agent surpasses the PID controller in terms of tracking accuracy, stability, adaptability, and generalizability. These findings highlight the potential of deep reinforcement learning algorithms in solving complex control tasks and facilitating the development of versatile and high-performance robots.

On the other hand, it is important to acknowledge the limitations of our study. Firstly, our control strategy assumes a specific robot-scooter system configuration, and the results may vary for different system designs or parameters. Another limitation lies in the accuracy of the simulation environment. Although efforts were made to create a realistic simulation, there might be slight discrepancies between the simulated and real-world behavior of the robot-scooter system.

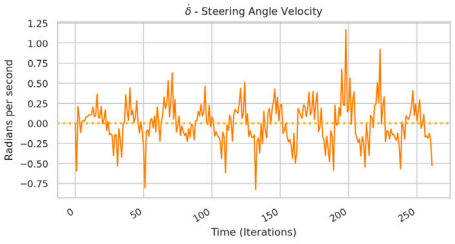
In terms of Real-world deployment, this problem involves considering three key aspects: training time, safety cost damages, and hardware costs.

- Sim-to-real and training time: This includes designing the training process and training our agent using our learning algorithm to fill the gap between simulation and the real environment.
- Safety Cost Damages: Ensuring safety is paramount in a real-world deployment. Implementing fail-safe mechanisms, conducting risk assessments, or following safety protocols in our problem is the biggest challenge. Additionally, consider the potential costs associated with accidents or damages during system operation and outline the strategies in place to mitigate these risks.
- Implementing our robotics system on a scooter requires acquiring our proper hardware components, including the robot, scooter, sensors, and actuators, which are quite costly.

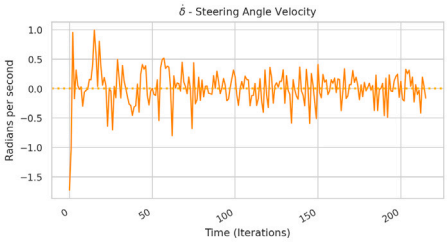
6. Conclusion

Two-wheeled vehicles are very difficult to control with robots, especially at low speeds. Humanoid robots, due to their high compatibility with human-made environments, have the potential to interact with the world better. The main objective of this paper is to propose a robust DRL algorithm for riding a commercially available electric scooter by a large-sized humanoid robot. In this regard, two different types of controllers (PID, and DRL algorithms) are proposed, developed, evaluated, and compared. Both controllers have been evaluated in balance and trajectory tracking tasks at different velocities and disturbances. In this research, we demonstrated both controllers are able to balance the robot-scooter system and track the proposed trajectories successfully. The Robot learned to tackle multiple turning velocity rates, resulting in stable balancing control behavior. We demonstrated that the proposed DRL controller, based on the PPO algorithm and simulated in the state-of-the-art NVidia's Isaac Gym environment, demonstrated an average improvement of 52% in comparison to the PID controller when evaluated at different speeds and steering velocities for balance and trajectory tracking tasks. The feasibility of the controller commands for steering velocity has been assessed using the real robot and scooter, and it has been demonstrated that the real robot is capable of realizing these commands to a certain extent.

In future work, we plan to carry out a comparative study with precisely matched optimization problems and performance specifications. By aligning both controllers' design and optimization processes, we aim to provide a more comprehensive and rigorous evaluation that directly compares their capabilities and performance under identical conditions, and also, it is intended to transition the simulation results to costly and challenging experiments on the real robot in a large testing environment.



(a) Best case steering velocity for the PID controller at 40 rad/s. Velocities are within 1.25 rad/s



(b) Best case steering velocity for the DRL agent at 40 rad/s. Velocities are within 1 rad/s.

Fig. 34. The best case steering velocity for the PID controller and DRL agent.

CRediT authorship contribution statement

Jacky Baltes: Conceptualization, Funding acquisition, Investigation, Methodology, Resources, Validation. **Guilherme Christmann:** Conceptualization, Data curation, Formal analysis, Methodology, Validation, Writing – original draft. **Saeed Saeedvand:** Conceptualization, Validation, Revision, Writing – review & editing.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Jacky Baltes reports equipment, drugs, or supplies was provided by gogoro.

Data availability

No data was used for the research described in the article.

Acknowledgments

This work is supported by Gogoro cooperation in Taiwan by providing the scooters for the experiments. Also, this work is financially supported Chinese Language and Technology Center of National Taiwan Normal University from The Featured Areas Research Center Program within the framework of the Higher Education Sprout Project by the Ministry of Education in Taiwan, and the Ministry of Science and Technology, Taiwan, under grants nos. MOST 108-2634-F-003-002, MOST 108-2634-F-003-003, and MOST 108-2634-F-003-004 (administered through Pervasive Artificial Intelligence Research Labs), as well as MOST 107-2811-E-003-503. We are grateful to the National Center for High-Performance Computing for the facilities to conduct this research.

References

Abd Elaziz, M., Dahou, A., Abualigah, L., Yu, L., Alshinwan, M., Khasawneh, A.M., Lu, S., 2021. Advanced metaheuristic optimization techniques in applications of deep neural networks: a review. Neural Comput. Appl. 1–21.

Abualigah, L., Diabat, A., Mirjalili, S., Abd Elaziz, M., Gandomi, A.H., 2021a. The arithmetic optimization algorithm. Comput. Methods Appl. Mech. Engrg. 376, 113609.

Abualigah, L., Yousri, D., Abd Elaziz, M., Ewees, A.A., Al-Qaness, M.A., Gandomi, A.H., 2021b. Aquila optimizer: a novel meta-heuristic optimization algorithm. Comput. Ind. Eng. 157, 107250.

Agushaka, J.O., Ezugwu, A.E., Abualigah, L., 2022. Gazelle optimization algorithm: A novel nature-inspired metaheuristic optimizer. Neural Comput. Appl. 1–33.

Astrom, K.J., Klein, R.E., Lennartsson, A., 2005. Bicycle dynamics and control: adapted bicycles for education and research. IEEE Control Syst. Mag. 25 (4), 26–47.

Baltes, J., Iverach-Brereton, C., Anderson, J., 2014. Human inspired control of a small humanoid robot in highly dynamic environments or jimmy darwin rocks the bongo board. In: Robot Soccer World Cup. Springer, pp. 466–477.

Baltes, J., Roux, U., Saeedvand, S., 2022. High-fidelity simulation of a humanoid robot driving an E-scooter using web technologies. In: 2022 International Conference on Advanced Robotics and Intelligent Systems (ARIS). IEEE, pp. 1–5.

gogoro, 2022. Gogoro. URL https://www.gogoro.com/.

Huang, C.-F., Tung, Y.-C., Lu, H.-T., Yeh, T.-J., 2018. Balancing control of a bicycle-riding humanoid robot with center of gravity estimation. Adv. Robot. 32 (17), 918–929.

Huang, C.-F., Tung, Y.-C., Yeh, T.-J., 2017. Balancing control of a robot bicycle with uncertain center of gravity. In: 2017 IEEE International Conference on Robotics and Automation (ICRA). IEEE, pp. 5858–5863.

Iverach-Brereton, C., Winton, A., Baltes, J., 2012. Ice skating humanoid robot. In: Conference Towards Autonomous Robotic Systems. Springer, pp. 209–219.

Jeong, J., Yang, J., Baltes, J., 2022. Robot magic show as testbed for humanoid robot interaction. Entertain. Comput. 40, 100456.

Joseph, S.B., Dada, E.G., Abidemi, A., Oyewola, D.O., Khammas, B.M., 2022. Metaheuristic algorithms for PID controller parameters tuning: Review, approaches and open problems. Heliyon e09399.

Kim, Y., Kim, H., Lee, J., 2015. Stable control of the bicycle robot on a curved path by using a reaction wheel. J. Mech. Sci. Technol. 29 (5), 2219–2226.

Klambauer, G., Unterthiner, T., Mayr, A., Hochreiter, S., 2017. Self-normalizing neural networks. Adv. Neural Inf. Process. Syst. 30, 971–980.

Le, T.N., Lundell, J., Abu-Dakka, F.J., Kyrki, V., 2022. Deformation-aware data-driven grasp synthesis. IEEE Robot. Autom. Lett. 7 (2), 3038–3045.

Lee, S., Ham, W., 2002. Self stabilizing strategy in tracking control of unmanned electric bicycle with mass balance. In: IEEE/RSJ International Conference on Intelligent Robots and Systems. vol. 3, IEEE, pp. 2200–2205.

Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D., 2015. Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971.

Madiouni, R., Bouallègue, S., Haggege, J., Siarry, P., 2019. Epsilon-multiobjective particle swarm optimization-based tuning of sensitivity functions for polynomial control design. Trans. Inst. Meas. Control 41 (13), 3688–3704.

Makoviychuk, V., Wawrzyniak, L., Guo, Y., Lu, M., Storey, K., Macklin, M., Hoeller, D., Rudin, N., Allshire, A., Handa, A., et al., 2021. Isaac gym: High performance gpu-based physics simulation for robot learning. arXiv preprint arXiv:2108.10470.

Messerschmidt, L., Engelbrecht, A.P., 2004. Learning to play games using a PSO-based competitive learning approach. IEEE Trans. Evol. Comput. 8 (3), 280–288.

Mnih, V., Badia, A.P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., Kavukcuoglu, K., 2016. Asynchronous methods for deep reinforcement learning. In: International Conference on Machine Learning. pp. 1928–1937.

Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M., 2013. Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602.

NVIDIA, 2022. Isaac gym. URL https://developer.nvidia.com/blog/introducing-isaac-gym-rl-for-robotics/.

Peters, J., Schaal, S., 2006. Policy gradient methods for robotics. In: 2006 IEEE/RSJ International Conference on Intelligent Robots and Systems. IEEE, pp. 2219–2225.

Rigatos, G., Siano, P., Selisteanu, D., Precup, R., 2017. Nonlinear optimal control of oxygen and carbon dioxide levels in blood. Intell. Ind. Syst. 3, 61–75.

Robotis, 2022. THORMANG3 e-manual. URL https://emanual.robotis.com/docs/en/platform/thormang3/introduction/.

Roman, R.-C., Precup, R.-E., Hedrea, E.-L., Preitl, S., Zamfirache, I.A., Bojan-Dragos, C.-A., Petriu, E.M., 2022. Iterative feedback tuning algorithm for tower crane systems. Procedia Comput. Sci. 199, 157–165.

Ruder, S., 2016. An overview of gradient descent optimization algorithms. arXiv preprint arXiv:1609.04747.

Saeedvand, S., Aghdasi, H.S., Baltes, J., 2019a. Robust multi-objective multi-humanoid robots task allocation based on novel hybrid metaheuristic algorithm. Appl. Intell. 49, 4097–4127.

Saeedvand, S., Jafari, M., Aghdasi, H.S., Baltes, J., 2019b. A comprehensive survey on humanoid robot development. Knowl. Eng. Rev. 34.

Saeedvand, S., Mandala, H., Baltes, J., 2021. Hierarchical deep reinforcement learning to drag heavy objects by adult-sized humanoid robot. Appl. Soft Comput. 110, 107601.

Schulman, J., Levine, S., Abbeel, P., Jordan, M., Moritz, P., 2015. Trust region policy optimization. In: International Conference on Machine Learning. pp. 1889–1897.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.

Sikander, A., Prasad, R., 2019. Reduced order modelling based control of two wheeled mobile robot. J. Intell. Manuf. 30 (3), 1057–1067.

Singhania, S., Kageyama, I., Karanam, V.M., 2019. Study on low-speed stability of a motorcycle. Appl. Sci. 9 (11), 2278.

Sünderhauf, N., Brock, O., Scheirer, W., Hadsell, R., Fox, D., Leitner, J., Upcroft, B., Abbeel, P., Burgard, W., Milford, M., et al., 2018. The limits and potentials of deep learning for robotics. Int. J. Robot. Res. 37 (4–5), 405–420.

Sutton, R.S., Barto, A.G., 2018. Reinforcement Learning: An Introduction. MIT Press.

Tanaka, Y., Murakami, T., 2004. Self sustaining bicycle robot with steering controller. In: The 8th IEEE International Workshop on Advanced Motion Control, 2004. AMC'04.. IEEE, pp. 193–197.

Utano, A., Yamakita, M., 2005. Automatic control of bicycles with a balancer. In: Proceedings, 2005 IEEE/ASME International Conference on Advanced Intelligent Mechatronics.. IEEE, pp. 1245–1250.

Verma, A., Meenpal, T., Acharya, B., 2022. Computational cost reduction of convolution neural networks by insignificant filter removal. Sci. Technol. 25 (2), 150–165.

Vu, N.K., Nguyen, H.Q., 2020. Balancing control of two-wheel bicycle problems. Math. Probl. Eng. 2020.

Wang, L.X., Eklund, J.M., Bhalla, V., 2012. Simulation & road test results on balance and directional control of an autonomous bicycle. In: 2012 25th IEEE Canadian Conference on Electrical and Computer Engineering (CCECE). IEEE, pp. 1–5.

Yamakita, M., Utano, A., Sekiguchi, K., 2006. Experimental study of automatic control of bicycle with balancer. In: 2006 IEEE/RSJ International Conference on Intelligent Robots and Systems. IEEE, pp. 5606–5611.

Zamfirache, I.A., Precup, R.-E., Roman, R.-C., Petriu, E.M., 2023. Neural network-based control using actor-critic reinforcement learning and grey wolf optimizer with experimental servo system validation. Expert Syst. Appl. 225, 120112.

Zhang, J., Zhang, Z., Han, S., Lü, S., 2022. Proximal policy optimization via enhanced exploration efficiency. Inform. Sci. 609, 750–765.